

DEVOPS E ARQUITETURA CLOUD

FASE 2 | AULA 01 -
**INTRODUÇÃO E
ARQUITETURA DO
KUBERNETES**

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
MERCADO, CASES E TENDÊNCIAS	14
O QUE VOCÊ VIU NESTA AULA?	15
REFERÊNCIAS.....	16

O QUE VEM POR AÍ?

Nessa aula, abordaremos de forma completa e detalhada a Introdução e Arquitetura do Kubernetes. Exploraremos os princípios básicos da plataforma, seus principais componentes, como o API Server, etcd, Scheduler, Controller Manager e kubelet, além da estrutura de nós e pods, comunicação entre serviços, mecanismos de escalabilidade e alta disponibilidade.

Também discutiremos boas práticas de **arquitetura** e **segurança**, oferecendo uma visão prática e aplicável ao dia a dia.

Nosso objetivo é fornecer uma base sólida para que os alunos e alunas compreendam os conceitos fundamentais do Kubernetes e saibam como aplicá-los de maneira eficiente. Cada tópico será explorado de forma objetiva, priorizando o entendimento claro dos princípios essenciais.

HANDS ON

Nesta etapa, serão disponibilizadas videoaulas práticas que guiarão o aluno e aluna na realização de exercícios voltados a criação, configuração e gerenciamento de clusters Kubernetes. Os vídeos estão estruturados para demonstrar, passo a passo, a aplicação prática dos conceitos abordados anteriormente na introdução e arquitetura do Kubernetes.

Durante as aulas, será oferecido autonomia para criar e gerenciar aplicações em contêineres utilizando ferramentas amplamente usadas no mercado, como o **Kubernetes Dashboard** e o **Lens**. Além disso, você aprenderá a implantar serviços, configurar comunicação entre componentes e aplicar políticas de segurança e escalabilidade.

Serão abordados **cases reais**, aplicando técnicas como: implantação de pods, criação de serviços (ClusterIP, NodePort, LoadBalancer), uso de volumes persistentes, escalabilidade com *Horizontal Pod Autoscaler* e adoção de boas práticas de arquitetura para garantir alta disponibilidade e desempenho.

SAIBA MAIS

Introdução e Arquitetura do Kubernetes

Começaremos explorando os fundamentos do Kubernetes. Kubernetes, conceitos essenciais para construir uma base sólida que facilitará o entendimento da arquitetura e do funcionamento dessa plataforma de orquestração de contêineres.

O que é Kubernetes?

O Kubernetes é uma plataforma open source para orquestração de contêineres, criada para automatizar grande parte das tarefas manuais relacionadas a implantação, administração e escalonamento de aplicações em contêineres.



Figura 1 - Kubernetes

Fonte: <https://www.cienciaedados.com/kubernetes-pods-nodes-containers-e-clusters> (2022)

Inicialmente criado pelo Google e atualmente mantido pela Cloud Native Computing Foundation (CNCF), o Kubernetes se consolidou como o padrão de mercado para orquestração de contêineres, sendo amplamente adotado em arquiteturas de microsserviços e computação em nuvem.

Por que usar Kubernetes?

- **Empacotamento Consistente:** as aplicações são empacotadas de forma que se comportem da mesma maneira em ambientes de desenvolvimento, teste e produção, eliminando o clássico problema do "funciona na minha máquina".

- **Abstração de Infraestrutura:** desenvolvedores não precisam se preocupar com servidores específicos. O Kubernetes se encarrega de alocar recursos e encontrar o melhor local para executar cada aplicação.
- **Escalabilidade sob Demanda:** o Kubernetes pode escalar automaticamente as aplicações — para mais ou para menos — com base na demanda real, otimizando recursos, desempenho e custos.
- **Plataforma Unificada:** permite gerenciar aplicações de forma centralizada em ambientes on-premises, na nuvem ou em arquiteturas híbridas, utilizando a mesma plataforma.
- **Ecossistema Robusto:** conta com uma vasta gama de ferramentas e extensões, além de uma comunidade ativa e forte suporte comercial de provedores líderes de mercado.
- **Automação do Ciclo de Vida:** desde a construção até o deploy e o monitoramento, todo o ciclo de vida da aplicação pode ser automatizado, aumentando a eficiência e reduzindo erros.
- **Portabilidade:** a mesma aplicação pode ser executada em diferentes provedores de nuvem (AWS, Azure, Google Cloud), ou em datacenters próprios, sem necessidade de alterações .

O que é um Cluster do Kubernetes?

Um cluster Kubernetes (ou K8s) é um conjunto de nós de computação (ou máquinas de trabalho), que trabalham juntos para executar aplicações empacotadas em contêineres. **A containerização** é um processo de implantação e execução de software que empacota o código da aplicação junto com todos os arquivos, bibliotecas e dependências necessários para que ela funcione de forma consistente em qualquer infraestrutura.

O Que é Um Contêiner do Kubernetes?

Um contêiner é uma unidade de software que empacota uma aplicação e todas as suas dependências, garantindo que ela seja executada de forma isolada e consistente em qualquer ambiente.

Componentes de um Cluster:

- **Control Plane:** o "escritório da gerência" que toma decisões.
- **Worker Nodes:** os "funcionários" que executam o trabalho real.
- **Rede:** a "comunicação interna" que conecta tudo.

Arquitetura do Kubernetes: Entendendo Seus Componentes Fundamentais

O Kubernetes é baseado em uma arquitetura composta por **componentes especializados**, que trabalham em conjunto para **automatizar a implantação, o escalonamento e a operação de aplicações em contêineres** de forma eficiente e resiliente.

Estrutura Básica do Cluster

Nós (Nodes) formam a infraestrutura física do cluster, divididos em:

Nós de Controle (Control Plane/Master Nodes):

- Gerenciam o estado global do cluster;
- Tomam decisões sobre agendamento;
- Armazenam configurações e estado;
- Não executam aplicações de usuário (boa prática).

Nós de Trabalho (Worker Nodes):

- Executam as cargas de trabalho (Pods);
- Recebem instruções do Control Plane.

Componentes do Control Plane

API Server - O Coração do Sistema: o **API Server** é o principal ponto de entrada para todas as operações dentro do cluster. Ele funciona como uma **"recepcionista central"**, coordenando a comunicação entre os usuários, componentes internos e ferramentas externas.

Sua função principal é processar requisições e todos os comandos kubectl:

- **Valida dados:** Verifica se as configurações estão corretas.
- **Autentica usuários:** Controla quem pode fazer o quê.
- **Persiste estado:** Salva tudo no etcd.
- **Expõe APIs:** Interface REST para ferramentas externas.

Etcd - O Banco de Dados Distribuído: o etcd é o repositório central de dados do Kubernetes. Ele atua como um **banco de dados chave-valor** altamente disponível e distribuído, responsável por armazenar **todo o estado do cluster**.

O que armazena:

- Estados de todos os objetos Kubernetes;
- Configurações de rede e segurança;
- Metadados e políticas;
- Histórico de mudanças.

Scheduler - O Planejador Inteligente: é responsável por decidir em qual nó do cluster cada pod será executado, garantindo eficiência e balanceamento de carga.

Processo de decisão:

- **Filtragem:** Remove nós que não atendem requisitos básicos.
- **Pontuação:** Avalia nós restantes com algoritmos de scoring.
- **Seleção:** Escolhe o nó com melhor pontuação.
- **Bind:** Agenda o Pod para execução.

Controller Manager - Os Supervisores Automáticos: é o componente que executa diversos **controllers**, responsáveis por garantir que o estado real do cluster se mantenha conforme o estado desejado definido na configuração.

Controllers principais:

- **Replication Controller / ReplicaSet:** garante que o número desejado de réplicas de um pod esteja sempre em execução.
- **Node Controller:** monitora o status dos nós (nodes) e gerencia ações quando um nó fica indisponível.
- **Service Account Controller:** gerencia contas de serviço e tokens de autenticação para os pods

Componentes dos Worker Nodes**kubelet - O Agente Local**

O kubelet é o agente que roda em cada nó de trabalho, responsável por garantir que os contêineres estejam funcionando conforme definido nas especificações do cluster

Responsabilidades detalhadas:

- **Comunicação:** registra o nó no cluster via API Server.
- **Gerenciamento de Pods:** baixa imagens, inicia contêineres, monitora saúde.
- **Health Monitoring:** executa probes e reporta status.
- **Resource Management:** aplica limits e requests de recursos.
- **Volume Management:** monta e gerencia armazenamento.

Kube-proxy - O Gerenciador de Rede

O kube-proxy é responsável por gerenciar a rede nos nós de trabalho garantindo a comunicação eficiente e o balanceamento de carga entre os pods.

Funcionalidades:

- **Load Balancing:** distribui o tráfego entre Pods de um Service.
- **Service Discovery:** mantém o mapeamento de Services para Pods.

- **Network Rules:** implementa regras de conectividade.
- **Health Tracking:** remove Pods não-saudáveis do balanceamento.

Container Runtime - O Executor

O Container Runtime é o componente responsável por executar e gerenciar contêineres em um nó, garantindo que eles sejam baixados, iniciados e gerenciados corretamente.

Responsabilidades:

- Baixar imagens de contêiner;
- Executar contêineres;
- Gerenciar ciclo de vida;
- Implementar isolamento de recursos.

Objetos Kubernetes Fundamentais

Pods - Unidades Básicas de Implantação

Os pods são a menor unidade executável no Kubernetes, com as seguintes características principais:

- Podem agrupar múltiplos contêineres;
- Compartilham recursos de rede e armazenamento;
- São sempre escalonados no mesmo nó;
- Funcionam como "unidades lógicas" para agrupamento de contêineres.

Estados do Pod:

- **Pending:** aguardando agendamento.
- **Running:** executando normalmente.
- **Succeeded:** terminou com sucesso.
- **Failed:** terminou com erro.

- **Unknown:** estado indeterminado.

Componentes de Gerenciamento

Serviços (Services): no Kubernetes, os Services atuam como uma abstração de rede que proporciona balanceamento de carga automático, oferece um ponto de acesso estável via DNS e permite o desacoplamento entre front-end e back-end.

Servidor de API: o API Server é o componente central do Kubernetes, responsável por processar todas as requisições de operação, validar e executar comandos no cluster e atuar como interface para ferramentas externas.

Mecanismos de Alta Disponibilidade

ReplicaSets: os ReplicaSets garantem a resiliência das aplicações através de:

- Manutenção de um número definido de réplicas.
- Recuperação automática de falhas.
- Escalabilidade horizontal simplificada.

Controladores de Ingress: gerenciam o tráfego externo para os serviços do cluster, oferecendo:

- Roteamento baseado em regras;
- Terminação SSL/TLS;
- Balanceamento de carga avançado.

Kubernetes: Capacidades Essenciais para Gestão de Aplicações em Contêineres

O Kubernetes oferece um ecossistema completo para a orquestração de contêineres, com funcionalidades avançadas que atendem aos requisitos de aplicações modernas:

Escalabilidade Automática

- Ajusta dinamicamente a quantidade de instâncias em execução conforme a demanda;
- Otimiza a utilização de recursos da infraestrutura;

- Permite redução de custos operacionais sem comprometer a performance;
- Suporta tanto escalonamento horizontal (mais réplicas) quanto vertical (mais recursos).

Balanceamento de Carga Inteligente

- Distribui automaticamente o tráfego entre os Pods disponíveis;
- Garante alta disponibilidade e estabilidade do sistema;
- Prevê e evita sobrecarga em componentes individuais;
- Oferece diferentes algoritmos de distribuição (Round Robin, Least Connections, etc.).

Mecanismos de Autocura

- Detecta e recupera automaticamente falhas em contêineres e Pods;
- Mantém o estado desejado declarado pelo usuário;
- Reduz significativamente o tempo de indisponibilidade;
- Reinicia contêineres falhos ou os realoca em nós saudáveis.

Service Discovery Integrado

- Simplifica a comunicação entre microsserviços;
- Oferece DNS interno para descoberta automática de serviços;
- Facilita a integração em arquiteturas distribuídas;
- Mantém o registro dinâmico de endpoints.

Gerenciamento de Configurações

- **ConfigMaps** armazenam configurações não sensíveis;
- **Secrets** gerenciam dados confidenciais com criptografia;
- Suporte nativo a variáveis de ambiente;
- Permite atualizações de configuração sem reconstruir imagens.

Estratégias Avançadas de Deploy

- Implementa atualizações contínuas (rolling updates);

- Permite rollback automático em caso de falhas;
- Suporta blue-green deployments e canary releases;
- Mantém a disponibilidade durante atualizações.

Gestão de Recursos Granular

- Define requests e limits para CPU e memória;
- Evita contenção de recursos entre aplicações;
- Permite priorização de cargas de trabalho críticas;
- Oferece métricas detalhadas de consumo.

Benefícios Estratégicos

- Redução de custos operacionais com otimização de recursos;
- Aumento da resiliência e disponibilidade das aplicações;
- Simplificação de operações complexas em ambientes distribuídos;
- Aceleração dos ciclos de desenvolvimento e entrega contínua.

Esta arquitetura de recursos integrados permite que as organizações gerenciem aplicações containerizadas de forma eficiente e escalável, desde ambientes de desenvolvimento até implementações críticas em produção.

MERCADO, CASES E TENDÊNCIAS

O Kubernetes consolidou-se como a principal referência mundial para orquestração de contêineres, conquistando adesão massiva em organizações de diversos segmentos.

Empresas emergentes optam por clusters compactos ou plataformas gerenciadas (EKS, GKE, AKS) para agilizar o time-to-market sem complexidade operacional significativa. Por outro lado, corporações estabelecidas implementam arquiteturas distribuídas em escala planetária, aproveitando funcionalidades sofisticadas de proteção, dimensionamento e monitoramento.

Estudos de Caso

Spotify: reestruturou sua infraestrutura em Kubernetes para dimensionar plataformas de música streaming com eficiência, assegurando estabilidade em cenários com milhões de conexões concorrentes.

Itaú Unibanco: no contexto nacional, a instituição financeira incorpora Kubernetes em sua evolução tecnológica, sustentando arquiteturas de microsserviços essenciais e abordagens de nuvem híbrida.

Movimentos Emergentes

Service Mesh e Arquitetura Zero-Trust: soluções como Istio e Linkerd expandem sua adoção para visibilidade operacional, roteamento inteligente e proteção nas interações entre microsserviços.

Inteligência Artificial e Automação: a aplicação de Machine Learning para antecipação de incidentes e otimização de auto-scaling emerge como vantagem estratégica.

Plataformas Internas para Desenvolvedores (IDPs): equipes de Platform Engineering desenvolvem abstrações sobre Kubernetes para proporcionar experiências intuitivas e autônomas aos desenvolvedores.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você conheceu os fundamentos do Kubernetes, uma plataforma open source para orquestração de contêineres criada pelo Google e mantida pela CNCF, que se consolidou como padrão de mercado para microserviços e computação em nuvem.

Foram abordados os conceitos essenciais da arquitetura Kubernetes, incluindo a estrutura de clusters com nós de controle e trabalho, pods como unidades básicas de implantação, serviços para abstração de rede, e componentes como ReplicaSets e controladores de Ingress.

REFERÊNCIAS

AWS. **O que é um cluster do Kubernetes?** Disponível em: <https://aws.amazon.com/pt/what-is/kubernetes-cluster/>. Acesso em: 01 set. 2025.

CIÊNCIA E DADOS. **Kubernetes: Pods, Nodes, Containers e Clusters.** Disponível em: <https://www.cienciaedados.com/kubernetes-pods-nodes-containers-e-clusters/>. Acesso em: 01 set. 2025.

RED HAT. **O que é Kubernetes?** Disponível em: <https://www.redhat.com/pt-br/topics/containers/what-is-kubernetes>. Acesso em: 01 set. 2025.

PALAVRAS-CHAVE

Kubernetes. Arquitetura. Componentes

EMENDADO



POS TECH